

## DreamLite Pipeline Math: From Prompt To Image

Think of the image pipeline as a chain of transformations. Each stage produces the numerical object that the next stage needs.

### 1. Text Becomes Numbers

Your prompt first becomes token embeddings:

prompt -> text\_encoder -> text embeddings

Inside the text encoder, many layers do:

$$y = Wx + b$$

Meaning:

- $x$  = input vector
- $W$  = learned weight matrix
- $b$  = learned bias
- $y$  = transformed vector

This turns words like:

"transparent turquoise glass warrior"

into dense numerical meaning vectors.

weight\_dtype affects how accurately  $W$ ,  $x$ ,  $b$ , and  $y$  are represented.

### 2. Attention Decides What Matters

Attention is how the model lets different parts of the prompt and image latent influence each other.

$$\text{attention} = \text{softmax}(QK^T / \sqrt{d})V$$

Rough intuition:

- $Q$  = queries, "what am I looking for?"
- $K$  = keys, "what information exists?"
- $V$  = values, "what content should I pass forward?"
- $QK^T$  = similarity scores
- $\text{softmax}(\dots)$  = converts scores into importance weights
- result = weighted mixture of useful information

So when generating the warrior, attention helps connect visual regions with concepts like:

glass  
turquoise  
running  
warrior  
isometric view

In float16, this part can become unstable because  $Q K^T$  can create large or tiny values before softmax.

### 3. UNet Predicts Noise

Diffusion starts from random noise in latent space.

latent\_0 = random noise from seed

At each step, the UNet predicts what noise should be removed:

noise\_prediction = UNet(latent, text\_embeddings, timestep)

Inside that UNet are lots of:

$y = Wx + b$

attention =  $\text{softmax}(Q K^T / \sqrt{d}) V$

So the prompt conditioning and current noisy image state interact repeatedly.

### 4. Scheduler Updates The Latent

Then the scheduler applies the denoising update:

next\_latent = scheduler\_step(latent, noise\_prediction)

Intuition:

current noisy image idea - predicted noise = cleaner image idea

This repeats for each inference step.

For 4 steps:

noise latent

-> denoise step 1

-> denoise step 2

- > denoise step 3
- > denoise step 4
- > final latent

## 5. VAE Turns Latent Into Pixels

The final latent is still not an image. It is a compressed representation.

```
image = VAE_decode(latent)
```

The VAE decoder converts that latent tensor into RGB pixels.

If earlier math produced bad values, or if VAE decode becomes unstable in float16, the output can become black.

### Full Chain

In one simplified flow:

prompt

- > text\_encoder:  $y = Wx + b$ , attention(...)
- > text\_embeddings

seed

- > random latent noise

text\_embeddings + latent

- > UNet noise prediction using  $Wx + b$  and attention

latent + noise\_prediction

- > scheduler\_step
- > repeated denoising

final latent

- > VAE\_decode
- > RGB image

weight\_dtype affects nearly every numeric operation in that chain. Same equations, different number system. float32 gives the equations more room and precision. float16 makes them cheaper but easier to break.

### Expanded Explanation: How $y = Wx + b$ And Attention Interact

The equation  $y = Wx + b$  is a learned transformation. It changes vectors from one representation into another. For example, in the text encoder it transforms token vectors. In the UNet it transforms latent feature vectors. It is how the model stores

learned visual and language knowledge.

Attention uses those transformed vectors. Q, K, and V are usually created by learned linear projections:

$$Q = W_q x + b_q$$

$$K = W_k \text{ context} + b_k$$

$$V = W_v \text{ context} + b_v$$

So  $y = Wx + b$  is not separate from attention. It is inside attention. The model first uses learned matrices to create Q, K, and V. Then attention compares Q and K, and mixes V.

For cross-attention inside the UNet:

- x is often the current noisy image latent features.
- context is often the text embeddings from the prompt.
- Q comes from the current image latent.
- K and V come from the prompt/text embeddings.

That means the noisy image asks, through Q: "What prompt information is relevant to this visual region right now?"

The prompt answers, through K and V: "Here are concepts like glass, warrior, turquoise, running, and isometric view."

Then:

$$\text{softmax}(Q K^T / \sqrt{d})$$

computes how strongly each visual position should attend to each prompt concept. Multiplying by V injects the relevant prompt information back into the image features.

So the two equations interact like this:

1. Linear layers create features and Q/K/V:

$$Q = W_q \text{ latent} + b_q$$

$$K = W_k \text{ text} + b_k$$

$$V = W_v \text{ text} + b_v$$

2. Attention matches image features to text concepts:

$$\text{attention} = \text{softmax}(Q K^T / \sqrt{d}) V$$

3. More linear layers transform the attention output:

$$y = W \text{ attention} + b$$

4. The UNet uses the result to predict noise:

```
noise_prediction = UNet(latent, text_embeddings, timestep)
```

Expanded Explanation: How Scheduler And VAE Interact

The scheduler and VAE operate at different parts of the pipeline.

The scheduler works in latent space. It does not create RGB pixels. It updates the compressed latent representation step by step.

At each denoising step:

```
noise_prediction = UNet(latent_t, text_embeddings, timestep_t)
```

```
latent_t_minus_1 = scheduler_step(latent_t, noise_prediction, timestep_t)
```

The scheduler asks: given the current latent and the noise predicted by the UNet, what should the next cleaner latent be?

After the final denoising step, you have:

```
final_latent
```

This final\_latent is not directly viewable as an image. It is a compact tensor representation. The VAE decoder then maps it into pixel space:

```
image = VAE_decode(final_latent)
```

So the scheduler and VAE interact sequentially:

1. Scheduler repeatedly refines the latent.
2. VAE decodes only the final refined latent into RGB pixels.

The VAE does not usually participate in every denoising step for text-to-image. It is mostly used at the end to decode the final latent. For image editing, the VAE may also be used at the beginning to encode the input image into latent space.

Precise simplified chain:

input prompt -> text embeddings

seed -> latent\_T

for each timestep t:

```
noise_prediction_t = UNet(latent_t, text_embeddings, t)
```

```
latent_t_minus_1 = scheduler_step(latent_t, noise_prediction_t, t)
```

```
final_latent = latent_0
```

```
image = VAE_decode(final_latent)
```

## Why Precision Matters

The same equations run under different number systems depending on `weight_dtype`.

`float16` uses less memory, but values can overflow, underflow, or lose detail. This can corrupt attention scores, noise predictions, scheduler updates, or VAE decoding.

`float32` uses more memory, but gives the equations more numeric room. That is why `float32` is often more stable and why `float16` can sometimes produce a black image.